

CineSeat — Teknik Rapor

Frontend ile backend'in nasıl konuştuğu, sistemin nasıl kurulduğu ve bu süreçte bulunup düzeltilen hatalar.

Bu belge ne için: kodu açmadan, sistemin nasıl çalıştığını ve nelerin neden değiştirildiğini anlamak için. Anlatılan her kod parçası projeden birebir alınmıştır; hangi dosyadan geldiği üstünde yazılıdır.

1. Sistemin şekli

CineSeat iki ayrı programdan oluşuyor. Bunlar aynı bilgisayarda çalışsalar bile **birbirinden tamamen bağımsız** iki süreçtir ve yalnızca ağ üzerinden, HTTP ile konuşurlar.

Parça	Teknoloji	Adres (geliştirme)	Görevi
Frontend	React + Vite	localhost:5173	Tarayıcıda çalışır, ekranı çizer, kullanıcıdan girdi alır
Backend	.NET 10 Web API	localhost:5207	İş kurallarını uygular, veritabanına yazar/okur
Veritabanı	PostgreSQL	localhost:5432	Kalıcı veri

Frontend'in veritabanıyla *hiçbir* teması yoktur. Tarayıcıda çalışan kod hiçbir zaman doğrudan SQL çalıştırmaz; her şey backend'in açtığı HTTP uçlarından geçer. Bu, güvenliğin temel taşıdır: tarayıcıdaki kod kullanıcının görebileceği ve değiştirebileceği bir şeydir, dolayısıyla ona güvenilmez.

2. Bağlantıyı kuran kod

"Frontend backend'e nasıl bağlanıyor" sorusunun cevabı tek bir satır değil, **dört ayrı parçanın birlikte çalışması**. Biri eksik olursa bağlantı kurulmaz.

2.1 Adres — nereye istek atılacağı

frontend/src/services/apiClient.js

```
const API_BASE_URL =  
  import.meta.env.VITE_API_BASE_URL || "http://localhost:5207/api";
```

Bağlantının başladığı yer burasıdır. Frontend'in backend'i bulabilmesi için adresini bilmesi gerekir. Bu adres tek bir yerde tanımlanmıştır; projedeki hiçbir başka dosya `localhost:5207` yazmaz.

`import.meta.env.VITE_API_BASE_URL` bir kaçış kapısıdır: canlıya çıkarken adres değişecektir ve o zaman kodu değil, yalnızca ortam değişkenini değiştirmek yeterli olur.

2.2 CORS — tarayıcının izin vermesi

Frontend 5173, backend 5207 portunda. Tarayıcı için bunlar **farklı köken** (origin) sayılır ve tarayıcı, bir sayfanın başka bir kökene istek atmasını **varsayılan olarak engeller**. Bu bir güvenlik önlemidir.

backend/Presentation/CineSeat.WebAPI/Program.cs

```
const string FrontendCorsPolicy = "FrontendCorsPolicy";
builder.Services.AddCors(options =>
{
    options.AddPolicy(FrontendCorsPolicy, policy =>
    {
        policy.WithOrigins("http://localhost:5173", "http://127.0.0.1:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});
```

Bu blok olmasaydı hiçbir şey çalışmazdı. Kod doğru, token doğru, veritabanı dolu olsa bile tarayıcı isteği göndermeden iptal ederdi. Backend'in "bu adresten gelen isteklere izin veriyorum" demesi şarttır.

Adres listesi sabit yazılmıştır. Vite başka bir porta düşerse (5173 doluysa 5174'e geçer) uygulama sessizce çalışmaz hale gelir — bu rapor hazırlanırken de bir kez yaşandı.

2.3 Tek giriş noktası — her isteğin geçtiği yer

Projede **hiçbir bileşen doğrudan fetch çağırmaz**. Her istek aşağıdaki fonksiyondan geçer:

frontend/src/services/apiClient.js

```

async function apiRequest(path, { method = "GET", body, headers } = {}) {
  const token = getStoredToken();

  const response = await fetch(`${API_BASE_URL}${path}`, {
    method,
    headers: {
      ...(body !== undefined ? { "Content-Type": "application/json" } : {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...headers,
    },
    body: body !== undefined ? JSON.stringify(body) : undefined,
  }).catch(() => {
    throw new ApiError(
      "Sunucuya ulaşılamıyor. Backend çalışıyor mu kontrol edin.",
      { status: 0, code: "NETWORK_ERROR" }
    );
  });

  if (!response.ok) {
    if (response.status === 401 && token && onUnauthorized) {
      onUnauthorized();
    }
    throw await toApiError(response);
  }

  if (response.status === 204) {
    return null;
  }

  return response.json();
}

```

Bu fonksiyon dört işi tek yerde yapar:

- **Adresi tamamlar** — çağırılan `/movies` yazar, buraya `http://localhost:5207/api/movies` olur.
- **Token'ı ekler** — oturum varsa `Authorization` başlığı kendiliğinden eklenir. Hiçbir servis dosyası token'ın varlığını bilmez.
- **Ağ hatasını çevirir** — backend kapalıysa `fetch` ham bir hata fırlatır; burada anlaşılır bir mesaja dönüşür.
- **204 durumunu ayırır** — silme işlemleri gövdesiz cevap döner; `response.json()` çağrılrsa çökerdi.

Dışarıya açılan yüzey dört satırdır; tüm servisler yalnızca bunları kullanır:

```

export const apiClient = {
  get: (path) => apiRequest(path),
  post: (path, body) => apiRequest(path, { method: "POST", body }),
  put: (path, body) => apiRequest(path, { method: "PUT", body }),
  del: (path) => apiRequest(path, { method: "DELETE" }),
};

```

2.4 Kimlik — token'ın hayat döngüsü

HTTP **durumsuzdur**: backend, arka arkaya gelen iki isteğin aynı kişiden geldiğini kendiliğinden bilmez. Bunu çözen şey her istekte taşınan imzalı bir belgedir — JWT.

Üretim (backend)

backend/Infrastructure/CineSeat.Infrastructure.Security/JwtTokenService.cs

```
var claims = new List<Claim>
{
    new(ClaimTypes.NameIdentifier, userId.ToString()),
    new(ClaimTypes.Name, username),
    new(ClaimTypes.Email, email),
    new(ClaimTypes.Role, roleName),
    new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
};

claims.AddRange(
    permissions.Select(p => new Claim(PermissionClaimTypes.Permission, p)));

var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
```

Token'ın içine kullanıcının kimliği, rolü ve **izinlerinin tamamı** yazılır. Sonra gizli anahtarla imzalanır. İmza sayesinde içerik kurcalanamaz: kullanıcı token'ı açıp okuyabilir ama değiştiremez — değiştirirse imza tutmaz.

Rol değil izin taşınıyor. "Bu kullanıcı Admin" demek yerine "bu kullanıcı `movie.manage`, `user.manage` ... yapabilir" deniyor. Böylece yeni bir yetki eklendiğinde kod değil veri değişir ve kısıtlı yetkili yönetici tanımlamak mümkün olur.

Saklama (frontend)

frontend/src/context/AuthProvider.jsx

```
const login = async (email, password) => {
    const loggedInUser = await authService.login(email, password);
    setIsSessionExpired(false);
    setUser(loggedInUser);
    sessionStorage.setItem("cineseat_user", JSON.stringify(loggedInUser));
    return loggedInUser;
};
```

Token, kullanıcı nesnesinin içinde `sessionStorage` 'a yazılır. `localStorage` değil: sekme kapanınca oturum düşün diye.

Doğrulama (backend)

backend/Presentation/CineSeat.WebAPI/Program.cs

```
options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer = true,
    ValidateAudience = true,
    ValidateLifetime = true,
    ValidateIssuerSigningKey = true,
    ValidIssuer = jwtSettings.Issuer,
    ValidAudience = jwtSettings.Audience,
    IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings.Key)),
    ClockSkew = TimeSpan.Zero
};
```

`ClockSkew = TimeSpan.Zero` bilinçli: varsayılan 5 dakikalık tolerans, süresi dolmuş bir token'ı 5 dakika daha geçerli sayar. Sıfırlandı.

Okuma (handler'lar)

backend/Infrastructure/CineSeat.Infrastructure/Services/CurrentUserService.cs

```
public long? UserId
{
    get
    {
        var raw = Principal?.FindFirstValue(ClaimTypes.NameIdentifier);
        return long.TryParse(raw, out var id) ? id : null;
    }
}

public bool HasPermission(string permission)
    => Principal?.HasClaim(PermissionClaimTypes.Permission, permission) ?? false;
```

Bu sınıf sahiplik kontrollerinin temelidir. "Bu rezervasyon senin mi" sorusunun cevabı istemciden değil, token'dan gelir. İstemci kendi kullanıcı numarasını gönderseydi, o numarayı değiştirip başkasının verisini okuyabilirdi.

2.5 Yetki kapısı

backend/Presentation/CineSeat.WebAPI/Program.cs

```
builder.Services.AddAuthorization(options =>
{
    foreach (var permission in PermissionNames.All)
    {
        options.AddPolicy(
            permission,
            policy => policy.RequireClaim(PermissionClaimTypes.Permission, permission));
    }
});
```

Her izin adı için otomatik bir politika üretilir. Bir uca yetki koymak tek satıra iner:

```
[Authorize(Policy = PermissionNames.MovieManage)]
[HttpPost]
public async Task<IActionResult> Create([FromBody] CreateMovieCommand command)
```

2.6 Hata sözleşmesi

Backend her hatada **aynı biçimde** cevap verir; frontend de bu biçimi kendi hata sınıflarına çevirir. İki taraf arasındaki bu anlaşma, sayfaların HTTP kodlarıyla uğraşmasını gereksiz kılar.

backend/Presentation/CineSeat.WebAPI/Middleware/ExceptionHandlingMiddleware.cs

```
try
{
    await _next(context);
}
catch (ValidationException ex)
{
    await WriteAsync(context, StatusCodes.Status400BadRequest, new
    {
        title = "Doğrulama hatası",
        status = StatusCodes.Status400BadRequest,
        errors = ex.Errors
    });
}
catch (NotFoundException ex) { /* 404 */ }
catch (ConflictException ex) { /* 409 */ }
```

Bu, uygulamadaki **tek try/catch**. Hiçbir controller ve handler hata yakalamaz; hepsi istisna fırlatır, çeviri burada yapılır.

frontend/src/services/apiClient.js

```
switch (response.status) {
    case 400:
    case 422: return new ValidationError(message);
    case 401: return new ApiError(message, { status: 401, code: "UNAUTHORIZED" });
    case 403: return new ForbiddenError(message);
    case 404: return new NotFoundError(message);
    case 409: return new ConflictError(message);
    default: return new ApiError(message, { status: response.status });
}
```

2.7 Oturum düştüğünde

Token'ın süresi dolduğunda kullanıcı "giriş yapmış" görünmeye devam eder ama her istek 401 döner. Bunu çözen bağ şu:

frontend/src/context/AuthProvider.jsx

```
useEffect(() => {  
  setUnauthorizedHandler(() => {  
    setIsSessionExpired(true);  
    clearSession();  
  });  
  
  return () => setUnauthorizedHandler(null);  
}, [clearSession]);
```

`apiClient` React'i hiç tanımaz — hook, context, router bilmez. Buna rağmen oturumun düştüğünü haber verebilir: `AuthProvider` açılırken kendi fonksiyonunu `apiClient` 'a kaydeder. Bu, iki katmanı birbirine bağımlı yapmadan haberleşmelerini sağlar.

3. Bir isteğin uçtan uca yolculuğu

Yönetim panelinde "yeni film ekle" düğmesine basıldığında olanlar, sırayla:

- 1 **Bileşen servisi çağırır.** Ekran `fetch` bilmez, yalnızca `movieService.create(values)` der.
- 2 **Servis komutu hazırlar.** Form değerleri backend'in beklediği alan adlarına çevrilir, sonra `apiClient.post("/movies", command)`.
- 3 **apiClient isteği kurar.** Adres tamamlanır, token `Authorization` başlığına eklenir, gövde JSON'a çevrilir.
- 4 **Tarayıcı CORS ön kontrolü yapar.** Token taşıyan bir POST, "basit istek" sayılmaz; tarayıcı önce `OPTIONS` gönderip izin sorar. Backend'in CORS politikası izin verirse asıl istek yola çıkar.
- 5 **Backend istisna sarmalayıcısına girer.** Boru hattının en dışı `UseExceptionHandler()`. Bundan sonrası ne fırlatırsa fırlatsın, istemci tutarlı bir JSON alır.
- 6 **Kimlik ve yetki.** `UseAuthentication` token'ın imzasını ve süresini doğrular, `UseAuthorization` `movie.manage` iznini arar. Yoksa handler'a hiç ulaşılmaz — 401 ya da 403.
- 7 **Controller komutu iletir.** Controller iş kuralı içermez; komutu aracıya verir.
- 8 **Doğrulama davranışı devreye girer.** Handler'dan önce çalışır; kurallar tutmuyorsa handler hiç çağrılmaz.
- 9 **Handler işi yapar.** Repository üzerinden veritabanına yazar.
- 10 **Cevap geri döner.** `apiClient` JSON'u çözer, servis arayüz şekline çevirir, ekran güncellenir.

Boru hattının sırası neden önemli

backend/Presentation/CineSeat.WebAPI/Program.cs

```
app.UseExceptionHandler(); // en dista: her şeyi sarmalı
// ...
app.UseCors(FrontendCorsPolicy);
app.UseAuthentication(); // once: sen kimsin
app.UseAuthorization(); // sonra: yetkin var mı
app.MapControllers();
```

CORS, kimlik doğrulamadan ÖNCE gelmek zorunda. Tarayıcının ön kontrol isteği (`OPTIONS`) hiçbir token taşımaz. CORS sonra gelseydi, bu istek kimlik doğrulamada 401 ile düşer ve tarayıcı asıl isteği hiç göndermezdi. Sonuç, hata ayıklaması zor bir "CORS hatası" olurdu.

Doğrulama davranışı

backend/Core/CineSeat.Application/Common/Behaviors/ValidationBehavior.cs


```

var results = await Task.WhenAll(
    _validators.Select(v => v.ValidateAsync(context, cancellationToken)));

var failures = results.SelectMany(r => r.Errors).Where(f => f is not null).ToList();

if (failures.Count > 0)
{
    // Handler hiç çağrılmaz; middleware bunu 400'e çevirir.
    throw new ValidationException(failures);
}

return await next();

```

Bu sayede handler'ların içi **tamamen iş kuralıdır**; "bu alan boş mu", "bu sayı pozitif mi" gibi kontroller handler'lara hiç girmez.

4. Backend'in katmanları

Backend dört katmana ayrılmış. Kural şudur: **bağımlılık her zaman içe doğrudur**. Dış katman iç katmanı tanır, tersi olmaz.

Katman	İçindekiler	Neyi tanımaz
Domain	Varlıklar, enum'lar	Hiçbir şeyi — en içteki katman
Application	Komutlar, sorgular, handler'lar, doğrulayıcılar, repository arayüzleri	Entity Framework'ü tanımaz
Persistence	DbContext, repository uygulamaları, migration'lar	HTTP'yi tanımaz
WebAPI	Controller'lar, middleware, kurulum	—

"Application EF'i tanımaz" kuralı somut bir sonuç doğurur: handler'lar `ToListAsync` gibi EF metotlarını çağıramaz. Bunun için bir arayüz konmuş:

backend/Infrastructure/CineSeat.Persistence/Data/EfAsyncQueryExecutor.cs

```

public sealed class EfAsyncQueryExecutor : IAsyncQueryExecutor
{
    public Task<List<T>> ToListAsync<T>(IQueryable<T> query, CancellationToken ct = default)
        => query.ToListAsync(ct);
    // CountAsync, AnyAsync, FirstOrDefaultAsync...
}

```

Bu kuralın pratik faydası rapordaki hatalardan birinde görüldü: veritabanına özgü bir hata kodunu (`SQLSTATE 23505`) yakalayıp anlamlı bir istisnaya çevirmek gerektiğinde, bunun *nereye* yazılacağı tartışmasızdı — EF'i yalnızca Persistence tanıdığı için oraya.

5. Bulunan ve düzeltilen hatalar

Aşağıdakiler bu süreçte ortaya çıkan **gerçek** hatalardır. Her biri ya canlı isteklerle ya da bir testle doğrulandı.

5.1 Ödemeye girer girmez sepetin temizlenmesi

Belirti: Kullanıcı ödeme sayfasına giriyor, anında sepeti boşalıyor ve sepete geri atılıyor.

Sebepe:

frontend/src/hooks/useCountdown.js – düzeltilmeden önce

```
const [seconds, setSeconds] = useState(initialSeconds);
```

useState kendisine verilen değeri **yalnızca ilk render'da** kullanır. Koltuk kilitleri asenkron alındığı için initialSeconds önce 0, kilitler gelince 600 oluyordu. Ama sayaç 0 'da kalıyor, kilit gelir gelmez "süre doldu" tetikleniyordu.

Düzeltilme:

```
const [syncedImage, setSyncedImage] = useState(initialSeconds);

if (initialSeconds !== syncedImage) {
  setSyncedImage(initialSeconds);
  setSeconds(initialSeconds);
}
```

Bu hatayı **bir test buldu**, elle deneme değil. Yalnızca gerçek akışta (asenكرون kilit + sayaç birlikte) ortaya çıkıyordu.

5.2 Sayfa boyutu uyumsuzluğu — üç kez tekrarlandı

Backend sayfa başına en fazla 100 kayıt veriyor:

```
RuleFor(x => x.PageSize).InclusiveBetween(1, 100);
```

Ön yüz ise 200, hatta 500 gönderiyordu. Sunucu 400 dönüyor, ekran sonsuza kadar "yükleniyor"da kalıyordu. Aynı hata **üç ayrı yerde** çıktı:

Nerede	Gönderilen	Sonuç
Sinema ve ilçe listeleri	200 / 500	400
Seans listesi (yönetim)	200	400
Film, yorum, favori, kullanıcı uçları	—	sınır yoktu: 999999 kabul ediliyordu

Sınıra inmek yeterli değildi. Bir sinemanın seansları 100'ü rahatlıkla aşabilir; `pageSize=100` yapmak veriyi sessizce kırpardı. Çözüm, sayfaları tek tek dolaşmak oldu:

`frontend/src/services/adminResource.js`

```
export async function fetchAllPages(basePath, params = {}) {
  const query = buildQuery(params);

  // Çağırılan belirli bir sayfa istediye ona karışılmaz.
  if (query.has("pageNumber") || query.has("pageSize")) {
    const result = await request();
    return Array.isArray(result) ? result : (result?.items ?? []);
  }

  const collected = [];
  query.set("pageSize", String(MAX_PAGE_SIZE));

  for (let pageNumber = 1; pageNumber <= MAX_PAGES; pageNumber += 1) {
    query.set("pageNumber", String(pageNumber));
    const result = await request();

    if (Array.isArray(result)) return result; // sayfalamayan uçlar

    const items = result?.items ?? [];
    collected.push(...items);

    const totalPages = Number(result?.totalPages) || 0;
    if (pageNumber >= totalPages || items.length === 0) break;
  }

  return collected;
}
```

`MAX_PAGES` üst sınırı bilinçli: sunucu tutarsız bir `totalPages` bildirirse bile döngü kapanır. Boş sayfa kontrolü ikinci güvencedir.

5.3 Kullanıcının gördüğü fiyattan farklı tutar ödemesi

Bilet tipi çarpanları iki tarafta ayırmıştı:

Taraf	Yetişkin	Öğrenci	Çocuk
Ön yüz	1.00	0.75	0.60
Backend (bağlayıcı)	1.00	0.75	0.50

Çocuk bileti ekranda 132 TL görünüp 110 TL tahsil ediliyordu. Bağlayıcı olan backend olduğu için ön yüz hizalandı ve kaynağın hangisi olduğu koda yazıldı:

`frontend/src/services/pricing.js`

```
/**
 * BAGLAYICI DEGER BACKEND'DE:
 * CreateReservationCommandHandler.TicketTypeMultiplier
 *
 * Buradaki kopya yalnızca on izleme için var (her tusa basista sunucuya
 * sormak mümkün değil), bu yüzden ikisi birebir aynı olmak ZORUNDA.
 */
const TICKET_MULTIPLIERS = {
  [TICKET_TYPE.ADULT]: 1.0,
  [TICKET_TYPE.STUDENT]: 0.75,
  [TICKET_TYPE.CHILD]: 0.5,
};
```

5.4 Ödemenin tümünden reddedilmesi — kampanya eşiği

Bu, ödemeyi kıran asıl hataydı ve belirtisi yanıltıcıydı: "indirim uygulanmıyor" değil, **rezervasyon hiç oluşmuyordu**.

Sepette N seans varsa **N ayrı rezervasyon** oluşuyor. Backend kampanya koşulunu *her rezervasyonun kendi* ara toplamına göre doğruluyor:

backend/.../CreateReservationCommandHandler.cs

```
if (subtotal < campaign.MinCartTotal)
    throw new ConflictException(
        $"Bu kampanya için sepet tutarı en az {campaign.MinCartTotal:0.00} olmalıdır.");
```

Ön yüz ise uygunluğu **sepetin tamamına** göre değerlendiriyordu:

Taraf	Gördüğü	Sonuç
Ön yüz	sepet $500 \geq 500$	indirim gösteriliyor
Backend	rezervasyon $250 < 500$	409 — ödeme düşüyor

Kampanya seçimi kalem bazına taşındı:

frontend/src/services/campaignService.js

```
export function planCampaignsPerItem(campaigns, items, user, calcItemSubtotal) {
  const lines = (items ?? []).map((item) => {
    const subtotal = calcItemSubtotal(item);
    const campaign = pickBestCampaign(campaigns, subtotal, user);

    return { item, campaign, subtotal,
      discount: calculateDiscount(campaign, subtotal) };
  });

  return {
    lines,
    discountTotal: lines.reduce((sum, line) => sum + line.discount, 0),
  };
}
```

Genel ders: istemci tarafındaki her uygunluk kontrolü, backend'in doğruladığı **aynı birim** üzerinde yapılmalı. Backend "sepet" diye bir şey bilmiyordu.

5.5 Beklenen durumların istisna sayılması

Üç ayrı hata aynı kökten geldi: **normal bir sonuç, hata gibi modellenmişti**. Bunun bedeli ağırdı — hata ayıklayıcı istisnada duraklayınca *tüm süreç donuyor* ve belirti her seferinde "sonsuz yükleniyor" oluyordu.

a) Zaten silinmiş kilidi bırakmak

Rezervasyon oluştuğunda kilit satırları aynı işlem içinde siliniyor. Ödeme sonrası temizlik yapan istemci, artık var olmayan kimlikleri bırakmaya çalışıyor ve her seferinde `NotFoundException` alıyordu.

Oysa DELETE **idempotent**ti: "bu kilit gitmiş olsun" demektir. Yoksa amaç zaten gerçekleşmiştir.

backend/.../ReleaseSeatCommandHandler.cs

```
// Kilit zaten yoksa amac gerceklesmis demektir, hata degil.
if (seatLock is null)
    return Unit.Value;

// Reddedilen istek de sessizce basarili doner: 404 dondurmek
// "bu kilit var ama senin degil" bilgisini sizdirirdi ve id deneyen
// biri gecerli kilitleri haritalayabilirdi. Kilide DOKUNULMUYOR.
if (seatLock.UserId != userId && !canManageShowtimes)
    return Unit.Value;

_write.HardDelete(seatLock);
await _write.SaveChangesAsync(cancellationTokens);
```

Durum	Cevap	Eylem
Kilit yok	204	—
Kilit senin	204	silinir
Kilit başkasının	204	dokunulmaz

b) Aynı kullanıcının kendi kendiyile çakışması

Handler "önce oku, sonra yaz" yapıyor. React'in geliştirme modunda efektleri iki kez çalıştırması (`StrictMode`) yüzünden aynı koltuk için **eş zamanlı iki istek** gidiyordu:

```
istek A: kilit var mi? -> hayir -> INSERT
istek B: kilit var mi? -> hayir -> INSERT    <- benzersiz dizin reddeder
```

Ama bu gerçek bir çakışma değildi — iki istek de aynı kullanıcıya aitti ve handler zaten "aynı kullanıcı yinelerse süreyi yeniler" diyordu. Yarış bu niyeti boşa çıkarıyordu.

backend/.../LockSeatCommandHandler.cs

```

try
{
    await _lockWrite.SaveAsync(cancellationToken);
    return ToDto(newLock);
}
catch (ConflictException)
{
    // YARIS: okuma ile yazma arasında baska bir istek ayni satiri ekledi.
    // Bu her zaman gercek bir cakisma DEGIL.
    //
    // Basarisiz ekleme izleyicide "Added" olarak duruyor; bir sonraki
    // kayitta tekrar denenmesin diye cikariliyor.
    _lockWrite.Detach(newLock);

    existingLock = await _lockRead.GetSingleAsync(
        sl => sl.ShowtimeId == request.ShowtimeId && sl.SeatId == request.SeatId,
        tracking: true, cancellationToken);

    if (existingLock is null) throw;
}
// ...asagidaki devralma mantigina dusuyor

```

Canlı doğrulama:

- Aynı kullanıcı, eş zamanlı 2 istek → **200 + 200**, ikisi de aynı kilit numarası
- Farklı kullanıcı, aynı koltuk → **409**

Düzeltilme yalnızca yarışı tolere ediyor, gerçek çakışmayı değil.

c) Benzersiz kısıt ihlalinin 500 dönmesi

Veritabanı doğru davranıp ikinci eklemeyi reddediyordu, ama `DbUpdateException` hiçbir yerde yakalanmadığı için kullanıcıya **"Sunucu hatası"** dönüyordu. Oysa bu beklenen bir durum: kayıt kapılmış.

backend/Infrastructure/CineSeat.Persistence/Repositories/WriteRepository.cs

```

private const string UniqueViolation = "23505";

public async Task<int> SaveAsync(Cancellation token cancellationToken = default)
{
    try
    {
        return await _context.SaveChangesAsync(cancellationToken);
    }
    catch (DbUpdateException exception)
    {
        when (exception.InnerException is PostgresException { SqlState: UniqueViolation })
        {
            throw new ConflictException(
                "Bu kayıt baska bir islem tarafından olusturulmus. Lutfen tekrar deneyin.");
        }
    }
}

```

Konum bilinçli: Application katmanı EF'i tanımıyor, Persistence tanıyor. Bu düzeltme yalnızca koltuk kilidini değil, benzersiz kısıtı olan *her* yolu (kullanıcı adı, e-posta, tür adı, rezervasyon numarası) 500 yerine 409'a

çeviriyor.

5.6 Seed'in tek kayıt yüzünden tümünden durması

backend/.../DbInitializer.cs - düzeltilmeden önce

```
if (await context.Cities.AnyAsync()) return;
```

Elle eklenmiş **tek bir şehir**, tüm ilçe ve sinema seed'ini kalıcı olarak bloke ediyordu — üstelik sessizce. Kontrol kayıt bazına indirildi: her kayıt "var mı, yoksa ekle" biçiminde ayrı değerlendiriliyor.

5.7 76 derleyici uyarısı

Uyarıların hepsi entity'lerdeki CS8618 idi. Hepsine körlemesine `= null!` yazmak yerine dört duruma ayrıldı:

Durum	Çözüm	Gerekçe
Zorunlu skaler	<code>required</code>	Derleme anında "set etmeyi unuttun" hatası
Zorunlu ilişki	<code>= null!</code>	EF dolduruyor; <code>?.</code> gürültüsü eklemes
Koleksiyon	<code>= new List<T>()</code>	Boş liste doğru cevap, <code>null</code> değil
İsteğe bağlı ilişki	<code>Campaign?</code>	Gerçekten null olabilir

Son satır bir **modelleme tutarsızlığını** açığa çıkardı: `Reservation.CampaignId` nullable'dı ama `Campaign` ilişkisi değildi. Kampanyasız rezervasyon tamamen geçerli bir durum.

`required` uygulanırken **tek bir derleme hatası çıkmadı** — yani her oluşturma noktası zaten tüm zorunlu alanları dolduruyormuş; artık bu derleyici tarafından güvenceye alınmış durumda.

6. Arayüz düzeltmeleri

6.1 İstatistik grafiği

Grafikteki asıl sorun boyut değil **biçimdi**: bilet adedi (onlu sayılar) ile gelir (binli sayılar) **aynı eksen**de gruplu çubuklardı. Farklı ölçekteki iki ölçü aynı eksende gösterildiğinde küçük olan görünmez kalır.

İki ayrı tek serili grafiğe bölündü. Ayrıca:

Şikâyet	Sebep	Çözüm
Çok büyük	Çubuk genişliği sınırsızdı	24 px sınır, tepesi 4 px yuvarlak
Beyaza dönüyor	Grafik kütüphanesinin varsayılan imleci	Nötr saydam vurgu
—	Renkler kütüphane varsayılanı	Tema başına ayrı seçildi

Renkler göz kararıyla seçilmedi; bir doğrulayıcıdan geçirildi (açıklık bandı, kroma tabanı, renk körlüğü ayrımı, kontrast):

Tema	Seri 1	Seri 2	Renk körlüğü ayrımı
Açık	#6b4790	#b3852b	ΔE 27.4
Koyu	#8765a3	#ad8838	ΔE 27.1

Eşik 8; ikisi de rahat geçiyor. Koyu tema için ilk seçilen sarı açıklık bandının dışında kaldı ve uygun ton bulunana kadar denendi.

frontend/src/components/admin/admin.css

```
/* Cubuk renkleri CSS'ten geliyor: grafik kutuphanesi fill'i oznitelik olarak yaziyor, oznitelikte var() cozulmuyor. CSS fill ozelligi onu geciyor ve tema degisince renk de degisiyor. */
.chart-bar-1 path { fill: var(--chart-series-1); }
.chart-bar-2 path { fill: var(--chart-series-2); }
```

6.2 Başlık gezintisi

"Hoşgeldin, X" tıklanmaz bir metindi ama bağlantıların arasında bağlantı gibi duruyordu; üstelik yanındaki "Profilim" ile aynı şeyi iki kez söylüyordu.

Karar: kaldırmak yerine işlevsel hale getirmek. İsim artık profilin kendi etiketi — kişisellik korundu, ölü metin gitti, tıklanabilir görünen şey gerçekten tıklanabilir oldu.

önce: ... Yönetim Hoşgeldin, Sistem Yöneticisi Profilim Çıkış Sepet 4 🍌
sonra: ... Yönetim | Sistem Yöneticisi Çıkış Sepet 0 *

Ayrıca site gezintisi ile hesap işlemleri tek düz sıradaydı ve peşi sıra yazılmış gibi okunuyordu. İnce bir ayırıcıyla iki gruba bölündü; ayırıcının iki yanına grup içi aralıktan belirgin şekilde geniş boşluk verildi.

7. Test altyapısı

Süreç başında backend'de **hiç test projesi yoktu**. Eklendi.

Neden gerçek veritabanı

Testler bellek içi sahte bir sağlayıcı yerine **ayrı bir gerçek PostgreSQL veritabanı** kullanıyor. Sebebi somut: ölçülmek istenen şeylerin bir kısmı (benzersiz kısıt, eş zamanlılık, `timestampz` davranışı) bellek içi sağlayıcıda zaten yok. Eş zamanlı kilit yarışı sahte bir veritabanında hiç ortaya çıkmazdı.

backend/Tests/CineSeat.IntegrationTests/CineSeatApiFactory.cs

```
private const string TestDatabase = "CineSeatDb_IntegrationTests";

protected override void ConfigureWebHost(IWebHostBuilder builder)
{
    builder.UseEnvironment("Testing");
    builder.UseSetting("ConnectionStrings:DefaultConnection", TestConnectionString);
}
```

Geliştirme veritabanına **dokunulmuyor**. Ortam `"Testing"` seçildi: referans verisi (rol, izin, admin) yüklenir ama demo katalog yüklenmez — böylece testler seed içeriği değiştiğinde kırılmaz, ihtiyaç duydukları kaydı kendileri kurar.

Kapsam

Dosya	Test	Ölçtüğü
PermissionMatrixTests	85	21 korumalı uç × (kimliksiz / izinsiz üye / yönetici)
PaginationLimitTests	17	Sekiz uçta sayfa boyutu sınırı
SeatLockConcurrencyTests	11	Eş zamanlı kilit, yarış, idempotency
ShowtimeConflictTests	7	Çakışma, temizlik payı sınırı
ReservationOwnershipTests	7	Sahiplik, iptal yetkisi
MovieArchivingTests	5	Arşivleme, geri alma

İzin matrisinde yönetici için "200 bekle" **denmedi**: gövdesiz bir POST 400, olmayan kayda PUT 404 döndürür. Ölçülen şey iş kuralı değil, isteğin **yetki duvarını geçip geçmediği** — dolayısıyla iddia "401 ve 403 değil".

8. Tekrar eden üç kalıp

Bulunan hataların neredeyse tamamı üç kalıptan geldi. Yeni bir hata ararken önce bunlara bakmak zaman kazandırır.

1 — Aynı kuralın iki yerde tutulması

Sayfa boyutu sınırı, bilet tipi çarpanları, kampanya eşiği. Üçü de hem ön yüzde hem backend'de tanımlıydı ve üçü de zamanla ayrıştı.

Önem: kuralın **bağlayıcı** olduğu tarafı yorumda açıkça yazmak ve bir testle sabitlemek. Kopya kaçınılmazsa (ön izleme için sunucuya sormak mümkün değil), hangisinin doğru olduğu belgelenmeli.

2 — İki tarafın farklı birim düşünmesi

Ön yüz "sepet" diye bir kavramla karar veriyordu; backend öyle bir şey bilmiyor, her rezervasyonu ayrı değerlendiriyordu.

Önem: istemcideki her uygunluk kontrolünü, backend'in doğruladığı **aynı birim** üzerinde yapmak.

3 — Beklenen durumun istisna olarak modellenmesi

Zaten silinmiş kilidi bırakmak, kendi kilidini yenilemek, benzersiz kısıt yarışını kaybetmek. Üçü de normal akıştı ama istisna fırlatıyordu.

Önem: istisna yazmadan önce "bu gerçekten bir hata mı, yoksa beklenen bir sonuç mu" diye sormak. Bedeli yüksek: hata ayıklayıcı istisnada duraklayınca tüm süreç donuyor ve belirti her seferinde "sonsuz yükleniyor" oluyordu — asıl sebeple hiç ilgisi olmayan bir görüntü.

Hata ayıklama notu: uygulama "sonsuz yükleniyor"da kalıyorsa, önce backend'in *yanıt verip vermediğine* bakın. Süreç hata ayıklayıcıda duraklamışsa hiçbir istek dönmez ve her ekran aynı şekilde takılır. Duraklama penceresindeki **istisna tipi** ve çağrı yığınının **en alt satırı** doğrudan sebebi gösterir.

9. Sayılarla durum

Ölçüm	Başlangıç	Şimdi
Derleme uyarısı (backend)		76 0
Derleme hatası		0 0
Backend entegrasyon testi	test projesi yoktu	126
Frontend testi	223	410 (47 dosya)
Lint hatası	0	0
Mock veriyle çalışan servis	7	0

Doğrulama komutları

```
cd backend
dotnet build CineSeat.slnx --no-incremental    # 0 uyarı, 0 hata
dotnet test  CineSeat.slnx                     # 126 / 126

cd frontend
npm run lint                                   # 0 hata
npm run test:run                              # 410 / 410
npm run build                                  # başarılı
```

Testler PostgreSQL ister. `dotnet test` çalıştırılırken yerelde PostgreSQL ayakta olmalı. Bu bilinçli bir tercih: eş zamanlılık ve veritabanı kısıtları sahte bir sağlayıcıda ölçülemez.

10. Bilinen açıklar

Aşağıdakiler **tespit edildi ama düzeltilmedi**. Her biri için gerekçe var.

Konu	Durum	Neden bekletildi
CancellationToken yaygınlığı	84 uçtan 48'i almıyor	İstemci bağlantıyı kesse bile sorgu sürüyor — kaynak israfı, doğruluk hatası değil. 48 imzayı değiştirmek büyük ve mekanik bir tur.
Alan adı tutarsızlığı	GetMoviesQuery Page , diğerleri PageNumber	Ön yüz buna göre yazıldığı için çalışıyor; tek başına düzeltmek ön yüzü kırar.
Panel paket boyutu	356 kB	Grafik kütüphanesinden geliyor. Zaten ayrı bir parçada; ilk açılışı etkilemiyor.
Rezervasyonun dağıtık olması	N seans = N istek	Araya giren hatada istemci telafi ediyor, ama tarayıcı kapanırsa telafi çalışmaz. Doğru çözüm çok seanslı tek uç — veri modelini etkiler.
CORS adresinin sabit olması	Yalnızca 5173	Vite başka porta düşerse uygulama sessizce çalışmaz. Geliştirme ortamıyla sınırlı.

11. Çalıştırma

```
# 1) Veritabanı ayakta olmalı (PostgreSQL, localhost:5432)

# 2) Backend
cd backend/Presentation/CineSeat.WebAPI
dotnet run                # http://localhost:5207
                          # migration'lar ve seed başlangıçta otomatik

# 3) Frontend
cd frontend
npm install
npm run dev                # http://localhost:5173
```

Frontend **5173** portunda açılmalı — backend'in CORS politikası bu adrese izin veriyor. Port doluysa Vite 5174'e düşer ve uygulama "Sunucuya ulaşılamıyor" hatası verir. Bu durumda 5173'ü boşaltın.

Yönetici hesabı (geliştirme): kullanıcı adı `admin` , parola `Admin123!`

CineSeat teknik raporu · `omer-cqrs-port` dalı

Ayrıntılı çalışma günlüğü: `docs/ENTEGRASYON_RAPORU.md` · Gözden geçirme: `docs/CODE_REVIEW_2.md`